

CS223 Computer Architecture & Organization

Static Scheduling for Multi-Cycle Pipelines



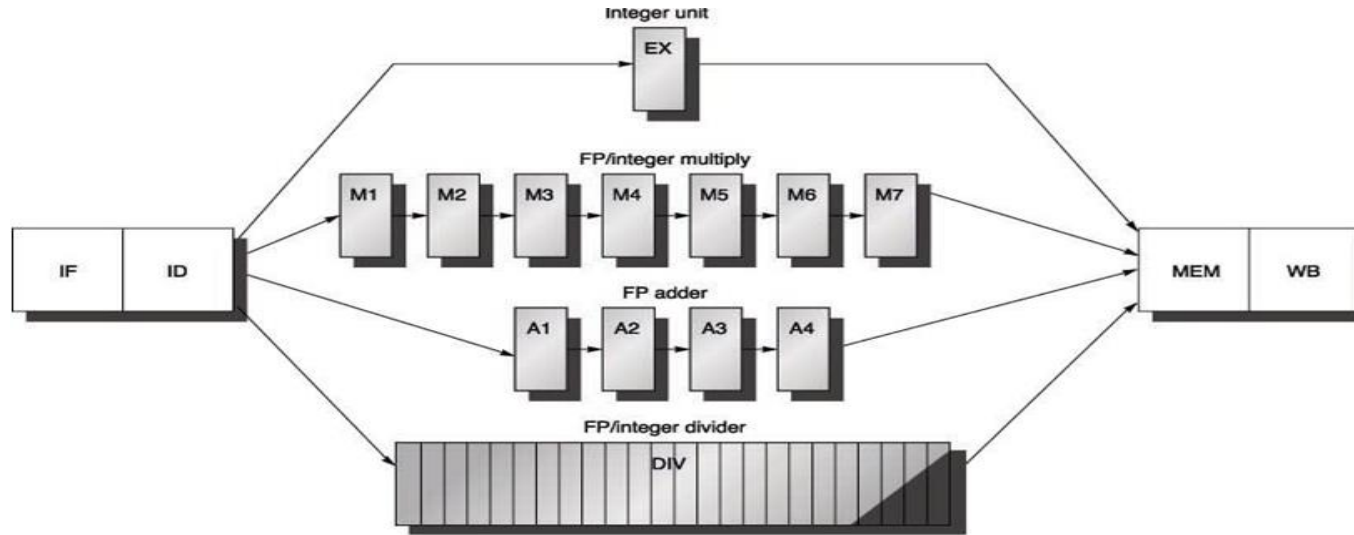
John Jose

Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Multi-cycle Operations



Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Latencies and initiation intervals for functional units.

Issues in Longer Latency Pipelines

- ❖ Since divide unit is not-pipelined – structural hazard
- ❖ Instruction have varying runtimes—more register writes/cycle
- ❖ WAW hazards possible
- ❖ Stalls for RAW hazards will be more.
- ❖ Single write port (Serialize completion) vs multiple write ports

Compiler Techniques for Exposing ILP

- ❖ Find and overlap sequence of unrelated instruction
- ❖ Pipeline scheduling
 - ❖ Separate dependent instruction from the source instruction by pipeline latency of the source instruction

❖ Example:

for (i=999; i>=0; i=i-1)

$x[i] = x[i] + s;$

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Pipeline Stalls

Lo: L.D F0,0(R1)

stall

ADD.D F4,F0,F2

stall

stall

S.D F4,0(R1)

DADDUI R1,R1,#-8

stall (assume integer load latency is 1)

BNE R1,R2,Lo

for (i=999; i>=0; i=i-1)

$x[i] = x[i] + s;$

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Pipeline Scheduling

Lo: L.D F0,0(R1)

stall

ADD.D F4,F0,F2

stall

stall

S.D F4,0(R1)

DADDUI R1,R1,#-8

stall (assume integer load latency is 1)

BNE R1,R2,Lo

Scheduled code:

Lo: L.D F0,0(R1)

DADDUI R1,R1,#-8

ADD.D F4,F0,F2

stall

stall

S.D F4,8(R1)

BNE R1,R2,Lo

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Loop Unrolling

❖ Loop unrolling

```
Lo: L.D F0,0(R1)
    ADD.D F4,F0,F2
    S.D F4,0(R1) ;drop DADDUI & BNE
    L.D F6,-8(R1)
    ADD.D F8,F6,F2
    S.D F8,-8(R1) ;drop DADDUI & BNE
    L.D F10,-16(R1)
    ADD.D F12,F10,F2
    S.D F12,-16(R1) ;drop DADDUI & BNE
    L.D F14,-24(R1)
    ADD.D F16,F14,F2
    S.D F16,-24(R1)
    DADDUI R1,R1,#-32
    BNE R1,R2,Lo
```

Scheduled code:

```
Lo: L.D F0,0(R1)
    DADDUI R1,R1,#-8
    ADD.D F4,F0,F2
    stall
    stall
    S.D F4,8(R1)
    BNE R1,R2,Lo
```

number of live registers ?

Loop Unrolling/Pipeline Scheduling

❖ Loop unrolling

Lo:L.D F0,0(R1)
ADD.D F4,F0,F2
S.D F4,0(R1)
L.D F6,-8(R1)
ADD.D F8,F6,F2
S.D F8,-8(R1)
L.D F10,-16(R1)
ADD.D F12,F10,F2
S.D F12,-16(R1)
L.D F14,-24(R1)
ADD.D F16,F14,F2
S.D F16,-24(R1)
DADDUI R1,R1,#-32
BNE R1,R2,**Lo**

❖ Scheduled unrolled loop:

Lo:L.D F0,0(R1)
L.D F6,-8(R1)
L.D F10,-16(R1)
L.D F14,-24(R1)
ADD.D F4,F0,F2
ADD.D F8,F6,F2
ADD.D F12,F10,F2
ADD.D F16,F14,F2
S.D F4,0(R1)
S.D F8,-8(R1)
DADDUI R1,R1,#-32
S.D F12,16(R1)
S.D F16,8(R1)
BNE R1,R2,**Lo**

Strip Mining

- ❖ Unknown number of loop iterations?
 - ❖ Goal: make k copies of the loop body Number of iterations = n
 - ❖ Generate pair of loops:
 - ❖ First executes $n \bmod k$ times
 - ❖ Second executes n / k times
 - ❖ Strip mining
- ❖ Example : Let $n=35$, $k=4$
 - ❖ Loop 1 execute 3 times
 - ❖ Loop 2 execute 8 times by unrolling 4 copies per iteration

Steps in Loop Unrolling and Scheduling

- ❖ Determine that unrolling the loop would be useful.
- ❖ Identify independency of loop iterations.
- ❖ Use different registers to avoid unnecessary constraints put in on same computations.
- ❖ Eliminate the extra test and branch instructions and adjust the loop termination and iteration code.
- ❖ Determine whether the loads and stores from different iterations are independent.
- ❖ Schedule the code, preserving any dependences needed to yield the same result as the original code.

Loop Unrolling & Pipeline Scheduling

❖ Limitations of loop unrolling:

- ❖ Code size limitations – I-cache miss
- ❖ Compiler limitations – register pressure

Dynamic Scheduling

- ❖ **Rearrange execution order of instructions to reduce stalls while maintaining data flow**
- ❖ **Advantages:**
 - ❖ Compiler doesn't need to have knowledge of micro-architecture
 - ❖ Handles cases where dependencies are unknown at compile time
- ❖ **Disadvantage:**
 - ❖ Substantial increase in hardware complexity

How dynamic scheduling works ?

- ❖ Limitation of simple pipelining.
 - ❖ In-order instruction issue and execution.
 - ❖ Instructions are issued in program order.
 - ❖ If an instruction is stalled in the pipeline, no later instructions can proceed.
- ❖ If instruction j depends on a long-running instruction i, currently in execution in the pipeline, then all instructions after j must be stalled until i is finished and j can execute.

DIV.D	F0, F2, F4
ADD.D	F10, F0, F8
SUB.D	F12, F8, F14



johnjose@iitg.ac.in
<http://www.iitg.ac.in/johnjose/>